

第 1 册 产品业务与技术架构

本册完整收录对应专题的原始内容，保留代码事实、现有限制与改进建议的区分。供开发维护和 AI 项目理解使用，不应直接发布为面向客户的售后政策。

本册专题：总览与阅读说明；产品定位与完整业务；技术架构与数据模型

SupportDeskAgent 项目知识库

本知识库面向项目开发者、毕业设计撰写者和需要接手开发的 AI，解释当前代码实际做了什么、为什么这样做、什么情况下会失败，以及用户误操作后有哪些恢复边界。它不是面向客户的售后政策，包含内部架构、权限和故障处理信息，不应直接发布到企业对客户开放的 RAG 知识库。

核对日期为 2026 年 9 月 30 日。依据是当前工作区源代码，包括已实现的订单售后预判功能，而不只依据旧聊天记录或 README。文中提到的配置值是代码默认值，不代表本机私有环境变量；本次没有读取或复制实际密钥、客户数据库内容、真实对话和内部业务资料。

这里的“已实现”表示在代码中有对应实现，并不等于已经生产部署、经过真实供应商全面验收或满足某项合规认证。“限制”说明现有边界；“建议”说明尚未落地的改进，不能写进简历作为已完成功能。函数名和路径用于定位源代码，不要求读者先理解 Python 或 TypeScript 才能阅读正文。

阅读目录

1. 产品定位与完整业务（第 1 册）：客户和企业分别做什么，订单、工单、人工接管怎样协作，哪些能力还没有实现。
2. 技术架构与数据模型（第 1 册）：前后端职责、技术栈实际使用情况、业务数据表、外键约束、Schema 与 ORM 的区别。
3. 聊天链路与 Agent 工作流（第 2 册）：一条消息怎样进入 Graph，全部节点和工具的真实关系，分支限制、确认授权和售后预判。
4. 记忆与会话恢复（第 2 册）：短期与长期记忆、消息裁剪、刷新、关闭浏览器、退出账号、切换企业及恢复人工会话。
5. 知识库检索与历史经验（第 3 册）：文档入库、分块预览、向量发布、检索、引用、案例脱敏、版本冲突和失效边界。
6. 认证权限与安全边界（第 4 册）：双认证、Cookie、密码、租户隔离、模型与工具权限、隐私和部署风险。
7. 可靠性与误操作处理（第 4 册）：幂等、事务、锁、错误回执、重复点击、网络超时、错选目标、并发编辑和安全恢复。
8. 前端页面与交互实现（第 4 册）：页面路由、Pinia、TanStack Query、网络边界、表单、轮询及未保存内容。
9. 接口与文件导航（第 5 册）：真实 API 清单、模块职责、入口定位、兼容文件和数据库迁移链。
10. 配置运行测试与维护（第 5 册）：启动、环境隔离、配置项、日志排错、测试范围、备份和维护建议。

如果只关心“误退出后怎么办”，直接阅读第 4、7、8 篇。如果要修改 Agent，先阅读第 3、4、6、7 篇；如果要修改 RAG，先阅读第 5、6 篇；如果要部署，重点阅读第 6、10 篇。后续发生代码变化，应更新

对应专题，而不是在聊天记录中维护另一套相互矛盾的结论。

必须先记住的事实

项目是多企业、双用户身份的客服与工单原型，不是独立运行的支付或订单交易平台。订单是客户可以自行创建的模拟数据；创建工单只登记诉求，不能代表退款、换货、维修已经审核或执行。

LangGraph 使用预先编写的分支与状态机进行流程编排；LLM 理解意图、提取信息、生成部分回答。它不是可以任意规划、任意访问数据库或执行任意工具的通用自主代理。部分确定性节点直接构造真实 Tool Call，这是受控工作流，不是伪造工具执行。

PostgreSQL 同时承担业务数据、官方 LangGraph 检查点、官方长期 Store 和 pgvector 向量存储。pgvector 是 PostgreSQL 扩展，不是另一套独立数据库。Redis、Celery、BM25 等虽然在 Python 依赖清单中，但当前主要运行链路没有接入这些能力。知识图谱也尚未实现，不能将数据库外键关系称为已落地的 GraphRAG。

已经保存到服务端的消息和未发送的输入草稿，是两种不同的数据。站内离开页面通常可以保留当前浏览器中的客户草稿；刷新、关闭浏览器和主动退出账号不能保证恢复草稿。已发送消息可重新登录后从历史会话中恢复，前提是服务器确实提交成功且数据库仍在。当前页面不会仅凭 localStorage 中的会话 ID 自动认领历史会话。

所有“结果未知”都需要核对业务回执，不能把超时等同于失败，也不能把模型说“完成了”等同于数据库完成。当前系统对危险重放采取保守阻止策略，这能降低重复写入风险，但也意味着部分中断需要管理员核对，而不是自动恢复。

文档保存与后续更新

这些文件保存在 `docs/project-knowledge/`。现有 `.gitignore` 忽略整个 `docs/`，本次遵循原约定，没有修改忽略规则，也没有提交 Git；因此这些文档在本地可用，但不会自动进入远程仓库。若希望团队共享，需要你另行决定是否调整这项约定。

文件内路径以仓库根目录为基准。本文档属于当前版本的自然语言解释，不是编译生成的接口规范；精确请求结构以 `backend/app/schema/`、路由代码及运行中的 OpenAPI 为准。对于旧教学文档与代码冲突的情况，以当前代码为依据。例如旧 README 曾残留 150 个分块的上限，而当前服务与 Schema 的上限是 300。

维护时至少核对五类变化：入口和字段是否变化，状态与权限是否变化，数据是否真正持久化，失败是否可能已经产生副作用，旧数据和检查点是否仍可读取。新增依赖、真实模型调用和对外发布文档都需要单独考虑，不因为更新知识库就自动进行。

产品定位与完整业务

项目解决的问题

SupportDeskAgent 将售后咨询中分散的工作串联到同一段会话：理解客户问题，查询企业政策和模拟订单，登记售后工单，查询进度，修改客户可编辑信息，必要时把原会话交给企业客服。客户不用先学习复杂表单；企业可以在工作台看客户、会话、工单与运行记录。

项目的“跨系统”目前主要体现为不同业务模块和外部模型服务的协作。订单、工单、账户、知识都在项目自己的数据库中，尚未对接真实电商 ERP、CRM、物流、支付或企业服务台。介绍项目时应说“面向跨系统售后任务的模块化编排原型”，不能说已经对接多家真实企业生产系统。

客户与企业不是同一身份

客户通过客户入口注册并登录，先选择企业再咨询。一个客户账户可以咨询多家企业，在每家企业下拥有不同的会话和模拟订单。客户的“我的工单”页面可以汇总本人跨企业的工单；进入某一家企业的 Agent 会话后，自然语言查询只针对这家企业。

企业员工通过企业编号、账号、密码登录企业工作台。创建企业入口会注册新企业并创建首个员工账户，不是申请加入已存在的企业。相同用户名可以分别存在于不同企业，但同企业内不能重复。当前没有员工邀请、部门组织、分级角色授权和企业资质认证流程。

企业员工查看的客户来自本企业业务关联，不能通过客户列表任意读取整个平台所有客户的会话。客户自己的身份来自登录会话，不能在聊天中说“我是某客户”来改变数据归属。

客户侧实际使用流程

客户登录后打开企业目录，选择企业进入咨询页。页面先显示固定的能力介绍，包含解答咨询、查询订单、创建工单和跟进工单四项。这段介绍是页面内容，不收费调用模型，不作为客户历史消息保存，也不代表人工客服在线。

客户输入文字后按 Enter 发送；Shift+Enter 换行，中文输入法选词期间的 Enter 不发送。消息最多 4000 字符。请求进行时不能再次发送，历史恢复期间也暂时禁止发送，减少重复请求和上下文错位。

“创建工单”“申请人工跟进”快捷按钮只把相应文字放入输入框，客户仍要发送。点击“我的模拟订单”或“查看我的工单”是页面导航，不会自动建单。快捷按钮会设置草稿，因此可能覆盖当前输入内容；这不是所有按钮都自动合并草稿的设计。

从订单页点击咨询此订单时，只在输入框为空的情况下预填订单编号。编号是定位线索，不是越权凭证，服务端仍查当前客户在当前企业的实际订单。

模拟订单的业务语义

每个客户在每家企业下默认补齐三个订单：家用机械设备 1299.00 元、已完成；无线键盘 199.00 元、已发货；年度维护服务 299.00 元、已支付。初始化不是依据“订单总数小于三”来判断，而是每个固定种子有唯一约束，因此客户自建订单不影响默认订单补齐。

客户可以创建商品名、金额、状态和可选签收日期组成的模拟订单。状态只有已支付、已发货、已完成。金额用十进制精确表示，必须大于零且不超过 99999999，最多两位小数；浏览器不是实际支付入口。

签收日期只允许已完成订单填写，且不能晚于北京时间的当天。为空表示未知。旧订单不根据创建时间推断签收日期。客户自己填写的模拟日期并不是物流凭据；对话中的自述日期只用于预判，不会悄悄更新数据库订单。

没有真实库存扣减、下单结算、物流履约、退款到账、订单状态自动同步和订单修改/删除界面。订单与工单通过外键关联，不将商品名和订单号塞进可修改的问题描述中。

创建工单有不同路径

普通问题建单适用于账号、技术问题等。Agent 判断客户明确要建单且标题、描述足够后，生成限定参数的工具调用，再由服务端创建。信息不足时只追问缺失信息。这条既有路径没有统一增加“确认提交”二次确认步骤，不能把新售后预判的安全门槛笼统说成所有创建路径都有。

订单关联建单先查数据库。多笔订单需要客户选择，唯一订单且基本诉求明确后可以登记。例如“这台机器故障，希望维修”已经可以形成问题，不要求客户解释技术根因。选择序号依赖上次展示列表，不能将“2”解释成任意数据库第二条记录。

新售后预判路径处理“我的无线键盘能退吗，可以的话帮我申请”这种同时需要规则判断和后续执行的请求。它查询订单、检索正式政策、核对签收日期和客户自述，展示申请预览，直到客户回复“确认提交”才建单。第一版商品名称精确限定为“无线键盘”，其他商品或非退货/退款诉求需要人工核实。它不是全商品售后审核引擎。

企业员工也可以从客户关联会话手动建单。员工界面创建来源为 manual，即使关联了 AI 会话也不能冒充 Agent 自动执行。后端要求关联可授权的客户会话，不能只填写随意客户名称生成脱离归属的工单。

工单的信息分层

工单有业务编号、标题、问题描述、售后诉求、影响或紧急情况说明、分类、状态、系统优先级、客户、企业、关联会话、可选订单、负责人名称和版本号。标题简要概括事项；描述记录实际问题；诉求记录希望退款、换货或维修等目标；影响说明记录客户主观陈述，不能自动等同于系统紧急优先级。

客户可以通过自然语言修改标题、描述、售后诉求、影响说明四项。状态、负责人、优先级、订单关联和企业归属不可通过这个客户工具修改。修改工单先选择候选，再展示当前字段与只读订单信息，再形成前后对照预览，客户明确回复“确认修改”才写入。只有一条候选也先请客户选择。

企业员工在工作台维护状态、优先级、负责人并追加内部处理备注。负责人目前是名称字符串，不是完整的员工任务分配与通知系统。内部备注不会通过客户的修改记录接口公开。客户的更新记录有特定标记，服务端只返回这些允许公开的活动。

工单状态的真实流转

待处理 open 可以转处理中 in_progress 或已关闭 closed。处理中可以转等待客户 waiting_customer、已解决 resolved 或已关闭。等待客户可以回处理中，或转已解决、已关闭。已解决可以重新转处理中，也可以关闭。已关闭没有状态回退路径。

因此“已解决”和“已关闭”并不是同一个状态；已关闭也不一定代表客户满意、退款到账或者故障真正修复。关闭可能只是业务处理终止，所以提炼历史经验仍要人工确认实际结果。

客户端禁止修改已关闭工单。企业侧状态机禁止把已关闭重新打开，但现有服务没有用“已关闭”统一禁止调整优先级、负责人或追加备注；说明权限时不能将“状态终态”说成“整条记录所有字段完全冻结”。

每次关键变化写入独立活动记录，修改字段同时更新 version 和更新时间。记录不能通过普通业务 API 覆盖或删除。它是应用层审计，不是具备防数据库管理员篡改能力的不可变审计平台。

人工接管流程

客户明确转人工，或 Agent 判断需要人工核实时，服务端记录申请时间和原因。企业“客户与会话”一级页面展示待接管队列，导航显示待办数量，员工可直接点击接管并回复。

待接管仍属于 active 会话。排队期间 AI 可以继续协助，不能说“转人工后 AI 立即停止”。只有员工成功接管，状态才变为 handed_off；之后客户发送的消息只保存给人工，不再调用模型和业务工具，也不会生成虚假的 AI 回复或 Agent 运行记录。

接管身份保存在数据库。其他员工可以查看本企业内容，但不能抢占已有接管者进行回复或恢复。当前接管者可以恢复智能客服；恢复后下条客户消息重新由 AI 处理，同时建立新的检查点代次，读取包括人工消息在内的业务历史，不延续之前未确认的工具授权。

没有自动分派、排队 SLA、客服在线探测、离线自动释放、跨员工转派和 WebSocket 推送。客服关闭页面或退出账号不会自动解除接管。因此“已申请人工”和“已有客服接通”必须区分；长期无人接待需要人工管理，而不是靠客户端猜测。

知识问答和历史经验

企业先录入公开政策或上传 TXT/Markdown，预览分块并确认发布，Agent 才能通过 RAG 检索。纯政策咨询走只读分支，不因为提到“退款”就自动退款或建单。

已解决或已关闭工单可以作为历史经验候选。员工在数据库候选中选择工单，获得脱敏的问题素材，补充已核实原因、实际解决办法和适用范围，再审核发布。它不是把所有已关闭工单自动全文塞入向量库，更不是把客户退款诉求直接当成企业实际退款结果。

目前能力的边界

已实现自然语言意图分流、受控工具调用、多轮订单与工单状态、数据库持久化、RAG、人工接管和基础运行追踪。尚未实现知识图谱、任意任务动态规划、自动退款支付、附件对话、多模态客服、语音、生产级工单 SLA、自动回复质量评测平台和完整数据生命周期管理。依赖清单或简历规划不能替代实现证据。

对应源码主要位于 `frontend/src/views/`、`backend/app/services/`、`backend/app/agent/workflows/`。精确调用逻辑见后续专题。

技术架构与数据模型

总体结构

项目采用前后端分离的单体后端架构。浏览器运行 Vue 应用，通过带 Cookie 的 HTTP 请求访问 FastAPI；后端访问 PostgreSQL，按需调用聊天模型与独立向量服务。Docker Compose 当前只定义 PostgreSQL 服务，不代表前后端已容器化，也不是微服务部署。

前端负责展示、输入、缓存和交互限制。后端负责身份、权限、业务状态、数据库事务与实际工具执行。模型是后端调用的外部能力，不能直接获得浏览器 Cookie、数据库连接或任意 SQL 权限。

实际技术栈

前端使用 Vue 3 单文件组件与 TypeScript。Vite 负责开发服务与生产构建；`vue-tsc` 负责 Vue 模板和 TypeScript 类型检查，不能只依赖开发页面打开来证明类型正确。Pinia 保存浏览器侧会话与草稿状态，Vue Router 划分客户和员工入口，TanStack Query 管理服务器数据缓存与异步请求，Element Plus 提供表单、表格、抽屉、提示等交互，Axios 统一 HTTP，Zod 在运行时校验响应。图标来自 `@lucide/vue`。

以当前 `package.json` 为准，声明 Vue 3.5.40、TypeScript 6.0.2、Vite 8.2.0、Pinia 4.0.2、Vue Router 5.2.0、TanStack Vue Query 5.10.4、Element Plus 2.14.3、Axios 1.19.0、Zod 4.4.3、pnpm 11.9.0。这是仓库版本快照，不是外部最新版本推荐。当前路由与依赖不使用 Next.js 或 Zustand；忽略文件中保留旧构建目录规则不意味着运行时还在使用它们。

后端使用 Python、FastAPI、Uvicorn、Pydantic、`pydantic-settings`、SQLAlchemy 异步会话、`psycopg`、Alembic 和 `structlog`。LangChain 提供模型适配、消息、工具和回调；LangGraph 提供有状态图与检查点；`langchain-openai` 通过兼容协议连接配置的 DeepSeek 模型和独立 Embedding 模型。使用兼容客户端不意味着实际聊天供应商必须是 OpenAI。

`requirements.txt` 固定了 LangGraph 1.2.10、`langgraph-checkpoint` 4.1.1、PostgreSQL `checkpoint` 包 3.1.1、`psycopg` 3.3.4、`psycopg-pool` 3.3.1；许多其他 Python 包没有固定版本。因此 `requirements` 能描述依赖需求，但还不是完整可复现锁文件。

Redis、Celery、SSE、MCP、LiteLLM、BM25、Ragas、Prometheus、OpenTelemetry、PDF/Office 解析以及 JWT/bcrypt 相关依赖虽然列在 `requirements` 中，不能据此宣称对应功能已启用。当前身份是数据库会话加 `scrypt`，当前聊天是普通 HTTP，当前 RAG 是文本分块加 `pgvector` 精确检索。

后端分层

`api/` 把 HTTP 路径、认证依赖、参数校验、异常状态码与服务连接起来。它不应成为所有业务逻辑的堆积处。

`schema/` 描述输入和输出协议。例如 `ChatRequest` 规定消息最大长度，`TicketChanges` 规定客户能修改的字段，`KnowledgePublish` 规定发布版本与审核确认。`Schema` 的作用是校验传输与业务边界，不负责建表。

`db/` 同时包含 ORM 实体和 `Repository`。ORM 实体描述表、字段、枚举、外键和关系；`Repository` 提供 SQL 查询与局部数据访问。`Repository` 的 `flush` 不一定意味着 `commit`，事务是否提交通常由 `Service` 决定。

`services/` 实现业务规则：认证、聊天请求幂等、会话持久化、工单状态机、订单初始化、人工接管、知识发布和客户偏好。需要多个表一起成功或回滚的动作主要放在这里。

`agent/` 负责意图结构、工具定义、模型配置、图节点与业务流程状态。`memory/` 描述可持续的会话状态和模型上下文窗口；`workflows/` 描述跨步骤处理；`graph.py` 注册节点与路由，不是数据库模型文件。

三种容易混淆的 model

`SQLAlchemy model` 是数据库对象，例如 `Ticket`。`Pydantic model` 是校验对象，例如 `AgentDecision`。`LLM model` 是外部语言模型的调用对象，例如 `ChatOpenAI`。这三者职责不同；名为 `order_agent_node` 的函数是 `LangGraph` 节点，不是另一个训练出来的大模型。

模型输出结构化 JSON 只代表可解析，不代表事实正确。系统还要检查工具名称、参数白名单、真实订单归属、当前版本及确认状态。权限不能依赖模型自称“已验证”。

业务表与关联

`companies` 保存企业 ID、编号、名称和启用状态。`customer_accounts` 保存全局客户账号、显示名、密码哈希和启用状态。`staff_accounts` 保存企业员工；企业 ID 与用户名构成唯一约束。

`login_sessions` 以令牌 SHA-256 摘要为主键，保存 audience、客户或员工主体、到期时间与创建时间。数据库约束要求 `customer` 会话只有 `customer_id`，`staff` 会话只有 `staff_id`，不能混合。

`auth_rate_limits` 保存认证来源摘要、次数和过期时间，不存原始 IP。

`conversations` 保存企业与客户、会话状态、创建更新时间、接管员工、申请时间、申请原因和 `memory_generation`。旧 `owner_key` 是历史兼容字段，不是现有认证凭证。企业和客户要么同时为空以兼容旧数据，要么同时存在；不会把旧无归属会话自动交给第一个注册客户。

`messages` 保存业务角色 `customer`、`agent`、`staff`、正文、会话 ID、时间和少量工具摘要。系统提示词并不是 `messages` 表中的第四种业务角色。消息时间使用实际插入时刻，读取按时间与 ID 升序稳定排列。完整工具参数和图运行过程主要在检查点中，不等同于业务聊天消息。

`demo_orders` 保存订单 ID、企业、客户、MO 编号、商品名、Numeric 金额、状态、种子标记、可空签收日期和创建时间。默认种子使用客户、企业和种子名称计算稳定 UUID；自建订单使用客户、企业和客户端请求键计算稳定主键。数据库还有客户、企业、种子唯一约束，避免并发重复初始化。

`tickets` 保存当前工单值。除了业务字段，还包含 `conversation_id`、`order_id`、`company_id`、`customer_id` 和 `version`。工单不是直接复制聊天记录，而是形成可追踪事项。编号采用 UTC 日期加 UUID 片段，展示用编号不是鉴权依据。

`ticket_activities` 保存创建、状态变化、优先级变化、分派和备注事件。客户修改复用 `note_added` 事件，但用 `customer_update` 标记区分；旧客户补充使用 `customer_comment`。客户读取接口按这些标记过滤，企业内部备注不会仅因为同属 `note_added` 就全部开放。

`chat_operations` 保存一次聊天请求的幂等状态、归属、内容指纹、`write_started`、工单或活动回执、完整成功响应或安全错误响应。它解决重试和副作用核对，不替代聊天历史，也不替代 LangGraph Checkpointer。

`agent_runs` 保存单次 Agent 运行的请求 ID、会话、状态、模型名、意图、优先级、需人工标志、判断摘要、最后工具、工单、耗时、安全错误摘要及 `steps`。没有专门保存原始客户问题的字段，但模型生成的判断摘要仍需按业务数据保护，不能视为天然无敏感信息。

`knowledge_documents` 保存企业、标题、原文、发布状态、模型指纹、片段数量、分块草稿、草稿版本、发布版本、分块警告和来源类型。经验文档额外关联来源工单和审核员工时间，每张来源工单最多一份经验文档。

`knowledge_chunks` 保存正式片段的正文、所属章节、顺序、版本和向量。原文、待发布草稿、线上正式片段是三份有不同职责的数据，不是一次编辑立即覆盖全部。片段向量列不固定单一维度，通过模型指纹限定检索范围。

官方 Checkpointer 与 Store 另行维护自己的 PostgreSQL 表。它们的表结构与迁移由相应库的 `setup` 管理，不能凭应用业务 Alembic 版本推断官方记忆表全部已经初始化。

数据库约束与事务边界

工单到会话、工单到订单都有包含企业和客户的联合外键。应用层即使未来漏写一次归属校验，数据库仍能阻止部分不一致关联。普通单字段外键只验证对象存在，联合外键进一步验证对象属于同一业务范围。

订单关联使用预加载、只读 `relationship`，避免在异步响应序列化阶段临时触发懒加载。客户看到的关联订单是只读结构，不通过改描述来更换真实订单。

业务事务并不是覆盖整次模型请求的一个巨型原子事务。客户消息先提交，工具业务写入另行提交，最终回复和运行结果再保存；检查点也走官方独立连接池。这样的分段可减少长事务，但要求用业务回执识别“已经建单、回复失败”这类中间状态。

部分表的外键声明具有删除级联或置空规则，不能推断项目已经提供删除用户、删除会话、撤回消息或一键清空历史功能。当前没有这类完整的业务生命周期界面与安全审计流程。

时间与数据类型

数据库多数时间戳带时区，以 UTC 写入或比较；浏览器通常用本地时区显示。签收日是 `date`，没有时分秒，未来日期检查与预判当天使用 `Asia/Shanghai`。金额后端使用 `Decimal`，JSON 响应通常是字符串，不能用 JavaScript 浮点值作为支付精度保证。

持久化消息不设“只存 40 条”的业务上限。40 是模型上下文窗口的默认条数约束之一；数据库历史和检查点增长需要独立的归档与容量治理，目前没有自动保留期策略。